

Java String Methods

Complete Laboratory Manual — 14 Core Methods with Programs, Outputs & Test Cases

1. charAt() Method

1. Aim

To write a Java program to demonstrate the use of the charAt() method to find the character present at a specified index in a String.

2. Definition

The charAt() method returns the character present at the specified index of a String. The index starts from 0.

Syntax: string.charAt(index);

3. Structure

```
String
|
+-- charAt(index)
    |
    +-- Returns character at specified index
    |
    +-- Return type: char
```

4. Program

```
class CharAtExample {
    public static void main(String[] args) {
        String str = "Hello Java";
        char ch = str.charAt(6);
        System.out.println("String: " + str);
        System.out.println("Character at index 6: " + ch);
    }
}
```

5. Sample Output

```
String: Hello Java
Character at index 6: J
```

6. Test Case Verification

Test Case	Input String	Index	Expected	Status
1	Hello	1	e	Pass
2	Java	0	J	Pass
3	Computer	3	p	Pass

7. Conclusion

Thus, the Java program was successfully executed, and the charAt() method was used to find the character at a specified index in a String.

2. toUpperCase() Method

1. Aim

To write a Java program to demonstrate the use of the toUpperCase() method to convert a String into uppercase letters.

2. Definition

The toUpperCase() method converts all lowercase characters in a String into uppercase characters. It returns a new String containing uppercase characters.

Syntax: string.toUpperCase();

3. Structure

```
String
|
+-- toUpperCase()
    |
    +-- Converts lowercase to uppercase
    |
    +-- Return type: String
```

4. Program

```
class UpperCaseExample {
    public static void main(String[] args) {
        String str = "hello java";
        String result = str.toUpperCase();
        System.out.println("Original String: " + str);
        System.out.println("Uppercase String: " + result);
    }
}
```

5. Sample Output

```
Original String: hello java
Uppercase String: HELLO JAVA
```

6. Test Case Verification

Test Case	Input String	Expected Output	Actual Output	Status
1	hello	HELLO	HELLO	Pass
2	java programming	JAVA PROGRAMMING	JAVA PROGRAMMING	Pass
3	computer	COMPUTER	COMPUTER	Pass

7. Conclusion

Thus, the Java program was successfully executed, and the toUpperCase() method was used to convert a String into uppercase letters.

3. toLowerCase() Method

1. Aim

To write a Java program to demonstrate the use of the toLowerCase() method to convert a String into lowercase letters.

2. Definition

The toLowerCase() method converts uppercase characters in a String into lowercase characters. It returns a new String containing lowercase characters.

Syntax: string.toLowerCase();

3. Structure

```
String
```

```

|
+-- toLowerCase()
|
+-- Converts uppercase to lowercase
|
+-- Return type: String

```

4. Program

```

class LowerCaseExample {
    public static void main(String[] args) {
        String str = "HELLO JAVA";
        String result = str.toLowerCase();
        System.out.println("Original String: " + str);
        System.out.println("Lowercase String: " + result);
    }
}

```

5. Sample Output

Original String: HELLO JAVA
 Lowercase String: hello java

6. Test Case Verification

Test Case	Input String	Expected Output	Actual Output	Status
1	HELLO	hello	hello	Pass
2	JAVA PROGRAMMING	java programming	java programming	Pass
3	COMPUTER	computer	computer	Pass

7. Conclusion

Thus, the Java program was successfully executed, and the toLowerCase() method was used to convert a String into lowercase letters.

4. equals() Method

1. Aim

To write a Java program to demonstrate the use of the equals() method to compare two Strings.

2. Definition

The equals() method compares two Strings based on their contents. It returns true if both Strings contain the same characters; otherwise, it returns false.

Syntax: string1.equals(string2);

3. Structure

```

String
|
+-- equals(String)
|
+-- Compares String contents
|
+-- Return type: boolean

```

4. Program

```

class EqualsExample {
    public static void main(String[] args) {
        String str1 = "Java";
    }
}

```

```

        String str2 = "Java";
        boolean result = str1.equals(str2);
        System.out.println("String 1: " + str1);
        System.out.println("String 2: " + str2);
        System.out.println("Strings are equal: " + result);
    }
}

```

5. Sample Output

```

String 1: Java
String 2: Java
Strings are equal: true

```

6. Test Case Verification

Test Case	String 1	String 2	Expected	Status
1	Java	Java	true	Pass
2	Hello	World	false	Pass
3	ABC	ABC	true	Pass

7. Conclusion

Thus, the Java program was successfully executed, and the equals() method was used to compare two Strings based on their contents.

5. equalsIgnoreCase() Method

1. Aim

To write a Java program to demonstrate the use of the equalsIgnoreCase() method to compare two Strings without considering their case.

2. Definition

The equalsIgnoreCase() method compares two Strings while ignoring differences between uppercase and lowercase letters.

Syntax: string1.equalsIgnoreCase(string2);

3. Structure

```

String
|
+-- equalsIgnoreCase(String)
    |
    +-- Compares contents without case
    |
    +-- Return type: boolean

```

4. Program

```

class EqualsIgnoreCaseExample {
    public static void main(String[] args) {
        String str1 = "Java";
        String str2 = "JAVA";
        boolean result = str1.equalsIgnoreCase(str2);
        System.out.println("String 1: " + str1);
        System.out.println("String 2: " + str2);
        System.out.println("Strings are equal: " + result);
    }
}

```

5. Sample Output

String 1: Java
String 2: JAVA
Strings are equal: true

6. Test Case Verification

Test Case	String 1	String 2	Expected	Status
1	Java	JAVA	true	Pass
2	Hello	hello	true	Pass
3	Java	Python	false	Pass

7. Conclusion

Thus, the Java program was successfully executed, and the `equalsIgnoreCase()` method was used to compare two Strings without considering their case.

6. concat() Method

1. Aim

To write a Java program to demonstrate the use of the `concat()` method to join two Strings.

2. Definition

The `concat()` method joins the specified String to the end of another String.

Syntax: `string1.concat(string2);`

3. Structure

```
String
|
+-- concat(String)
    |
    +-- Joins two Strings
    |
    +-- Return type: String
```

4. Program

```
class ConcatExample {
    public static void main(String[] args) {
        String str1 = "Hello ";
        String str2 = "Java";
        String result = str1.concat(str2);
        System.out.println("First String: " + str1);
        System.out.println("Second String: " + str2);
        System.out.println("Combined String: " + result);
    }
}
```

5. Sample Output

First String: Hello
Second String: Java
Combined String: Hello Java

6. Test Case Verification

Test Case	String 1	String 2	Expected Output	Status
1	Hello	Java	Hello Java	Pass

Test Case	String 1	String 2	Expected Output	Status
2	Good	Morning	Good Morning	Pass
3	Computer	Science	Computer Science	Pass

7. Conclusion

Thus, the Java program was successfully executed, and the concat() method was used to join two Strings.

7. substring() Method

1. Aim

To write a Java program to demonstrate the use of the substring() method to extract a portion of a String.

2. Definition

The substring() method returns a part of a String from the specified starting index.

Syntax: string.substring(beginIndex);

3. Structure

```
String
|
+-- substring(beginIndex)
    |
    +-- Extracts part of String
    |
    +-- Return type: String
```

4. Program

```
class SubstringExample {
    public static void main(String[] args) {
        String str = "Programming";
        String result = str.substring(3);
        System.out.println("Original String: " + str);
        System.out.println("Substring: " + result);
    }
}
```

5. Sample Output

```
Original String: Programming
Substring: gramming
```

6. Test Case Verification

Test Case	Input String	Index	Expected	Status
1	Programming	3	gramming	Pass
2	Computer	4	uter	Pass
3	Hello	2	llo	Pass

7. Conclusion

Thus, the Java program was successfully executed, and the substring() method was used to extract a portion of a String.

8. indexOf() Method

1. Aim

To write a Java program to demonstrate the use of the indexOf() method to find the position of a character in a String.

2. Definition

The indexOf() method returns the index of the first occurrence of the specified character or String.

Syntax: string.indexOf(character);

3. Structure

```
String
|
+-- indexOf()
    |
    +-- Finds first occurrence
    |
    +-- Return type: int
```

4. Program

```
class IndexOfExample {
    public static void main(String[] args) {
        String str = "Hello";
        int index = str.indexOf('l');
        System.out.println("String: " + str);
        System.out.println("Index of 'l': " + index);
    }
}
```

5. Sample Output

```
String: Hello
Index of 'l': 2
```

6. Test Case Verification

Test Case	Input String	Character	Expected	Status
1	Hello	l	2	Pass
2	Java	a	1	Pass
3	Computer	p	3	Pass

7. Conclusion

Thus, the Java program was successfully executed, and the indexOf() method was used to find the first occurrence of a character in a String.

9. lastIndexOf() Method

1. Aim

To write a Java program to demonstrate the use of the lastIndexOf() method to find the last occurrence of a character in a String.

2. Definition

The lastIndexOf() method returns the index of the last occurrence of the specified character or String.

Syntax: `string.lastIndexOf(character);`

3. Structure

```
String
|
+-- lastIndexOf()
    |
    +-- Finds last occurrence
    |
    +-- Return type: int
```

4. Program

```
class LastIndexOfExample {
    public static void main(String[] args) {
        String str = "Hello";
        int index = str.lastIndexOf('l');
        System.out.println("String: " + str);
        System.out.println("Last index of 'l': " + index);
    }
}
```

5. Sample Output

```
String: Hello
Last index of 'l': 3
```

6. Test Case Verification

Test Case	Input String	Character	Expected	Status
1	Hello	l	3	Pass
2	Java	a	3	Pass
3	Banana	a	5	Pass

7. Conclusion

Thus, the Java program was successfully executed, and the `lastIndexOf()` method was used to find the last occurrence of a character in a String.

10. replace() Method

1. Aim

To write a Java program to demonstrate the use of the `replace()` method to replace characters in a String.

2. Definition

The `replace()` method replaces all occurrences of a specified character with another character.

Syntax: `string.replace(oldChar, newChar);`

3. Structure

```
String
|
+-- replace()
    |
    +-- Replaces characters
    |
    +-- Return type: String
```

4. Program


```

class ReplaceExample {
    public static void main(String[] args) {
        String str = "Hello";
        String result = str.replace('l', 'p');
        System.out.println("Original String: " + str);
        System.out.println("After Replacement: " + result);
    }
}

```

5. Sample Output

Original String: Hello
After Replacement: Heppo

6. Test Case Verification

Test Case	Input String	Replacement	Expected	Status
1	Hello	l → p	Heppo	Pass
2	Java	a → o	Jovo	Pass
3	Hello	H → Y	Yello	Pass

7. Conclusion

Thus, the Java program was successfully executed, and the replace() method was used to replace characters in a String.

11. trim() Method

1. Aim

To write a Java program to demonstrate the use of the trim() method to remove leading and trailing spaces from a String.

2. Definition

The trim() method removes leading and trailing whitespace from a String.

Syntax: string.trim();

3. Structure

```

String
|
+-- trim()
    |
    +-- Removes leading and trailing spaces
    |
    +-- Return type: String

```

4. Program

```

class TrimExample {
    public static void main(String[] args) {
        String str = "  Hello Java  ";
        String result = str.trim();
        System.out.println("Original String: " + str);
        System.out.println("After trim(): " + result);
    }
}

```

5. Sample Output

Original String: Hello Java
After trim(): Hello Java

6. Test Case Verification

Test Case	Input String	Expected Output	Actual Output	Status
1	Hello	Hello	Hello	Pass
2	Java	Java	Java	Pass
3	Computer	Computer	Computer	Pass

7. Conclusion

Thus, the Java program was successfully executed, and the trim() method was used to remove leading and trailing spaces from a String.

12. contains() Method

1. Aim

To write a Java program to demonstrate the use of the contains() method to check whether a String contains a specified sequence of characters.

2. Definition

The contains() method checks whether a specified sequence of characters exists in a String. It returns true if found; otherwise, it returns false.

Syntax: string.contains(sequence);

3. Structure

```
String
|
+-- contains()
    |
    +-- Checks whether text exists
    |
    +-- Return type: boolean
```

4. Program

```
class ContainsExample {
    public static void main(String[] args) {
        String str = "I love Java";
        boolean result = str.contains("Java");
        System.out.println("String: " + str);
        System.out.println("Contains Java: " + result);
    }
}
```

5. Sample Output

```
String: I love Java
Contains Java: true
```

6. Test Case Verification

Test Case	Input String	Search Text	Expected	Status
1	I love Java	Java	true	Pass
2	Hello World	Java	false	Pass
3	Computer Science	Science	true	Pass

7. Conclusion

Thus, the Java program was successfully executed, and the `contains()` method was used to check whether a specified sequence of characters exists in a String.

13. startsWith() Method

1. Aim

To write a Java program to demonstrate the use of the `startsWith()` method to check whether a String starts with a specified sequence of characters.

2. Definition

The `startsWith()` method checks whether a String begins with the specified prefix. It returns `true` if the String starts with the prefix; otherwise, it returns `false`.

Syntax: `string.startsWith(prefix);`

3. Structure

```
String
|
+-- startsWith()
    |
    +-- Checks starting characters
    |
    +-- Return type: boolean
```

4. Program

```
class StartsWithExample {
    public static void main(String[] args) {
        String str = "Java Programming";
        boolean result = str.startsWith("Java");
        System.out.println("String: " + str);
        System.out.println("Starts with Java: " + result);
    }
}
```

5. Sample Output

```
String: Java Programming
Starts with Java: true
```

6. Test Case Verification

Test Case	Input String	Prefix	Expected	Status
1	Java Programming	Java	true	Pass
2	Hello World	Java	false	Pass
3	Computer Science	Computer	true	Pass

7. Conclusion

Thus, the Java program was successfully executed, and the `startsWith()` method was used to check whether a String starts with a specified prefix.

14. endsWith() Method

1. Aim

To write a Java program to demonstrate the use of the `endsWith()` method to check whether a `String` ends with a specified sequence of characters.

2. Definition

The `endsWith()` method checks whether a `String` ends with the specified suffix. It returns `true` if the `String` ends with the suffix; otherwise, it returns `false`.

Syntax: `string.endsWith(suffix);`

3. Structure

```
String
|
+-- endsWith()
    |
    +-- Checks ending characters
    |
    +-- Return type: boolean
```

4. Program

```
class EndsWithExample {
    public static void main(String[] args) {
        String str = "Java Programming";
        boolean result = str.endsWith("Programming");
        System.out.println("String: " + str);
        System.out.println("Ends with Programming: " + result);
    }
}
```

5. Sample Output

```
String: Java Programming
Ends with Programming: true
```

6. Test Case Verification

Test Case	Input String	Suffix	Expected	Status
1	Java Programming	Programming	true	Pass
2	Hello World	Java	false	Pass
3	Computer Science	Science	true	Pass

7. Conclusion

Thus, the Java program was successfully executed, and the `endsWith()` method was used to check whether a `String` ends with a specified suffix.